Lecture 5/6 Crash Study Guide

Concurrency - Mutual Exclusion - Synchronization

Read this first

You do not need to memorize every slide. You need to understand why shared data breaks, what mutual exclusion guarantees, and how each mechanism enforces it.

For the exam, prioritize: race condition, critical section, mutual exclusion requirements, test_and_set / compare_and_swap, mutex vs semaphore, producer-consumer, readers-writers, dining philosophers, monitors, condition variables, and message passing.

Time left	What to do
First 30 min	Study Sections 1-4: race conditions, critical section, requirements, hardware locks.
Next 50 min	Study Sections 5-6: semaphores + producer/consumer + readers/writers + dining philosophers.
Next 35 min	Study Sections 7-8: monitors + condition variables + message passing.
Final 20 min	Answer the likely exam questions and flashcards at the end.

0. The whole lecture in one mental picture

The core idea

Concurrency means several processes/threads are active in overlapping time. The danger is shared data/resources. If access is uncontrolled, the final result depends on timing, so the same program can produce different answers on different runs.

The cure is synchronization. The main form here is mutual exclusion: allow only one process/thread at a time inside the critical section for the same shared resource.

- Race condition = final result depends on execution order/timing.
- Critical resource = non-shareable resource, such as a printer or shared variable.
- Critical section/critical region = code that accesses the critical resource.
- Entry section = code that asks permission to enter the critical section.
- Exit section = code that releases permission after the critical section.
- Atomic operation = appears indivisible; no other process can observe an intermediate state.
- Deadlock = processes wait forever for each other.
- Starvation = one process waits indefinitely even though progress is happening elsewhere.

Concept	One-line exam answer
Concurrency	Managing the interaction of multiple processes/threads that may execute in overlapping time.
Synchronization	Coordinating process/thread activities so shared data/resources remain correct.
Mutual exclusion	Only one process/thread may access a critical resource at a time.
Critical section problem	Design an entry/exit protocol so cooperating processes safely share a resource.

Semaphore	Integer synchronization variable accessed only by atomic wait and signal operations.
Monitor	High-level language construct that gives mutual exclusion automatically inside monitor procedures.
Message passing	Processes communicate and synchronize using send and receive primitives.

1. Concurrency and why it creates problems

- OS design manages multiple processes and threads under multiprogramming, multiprocessing, and distributed processing.
- Concurrency covers communication, resource sharing/competition, synchronization, and CPU-time allocation.
- Concurrent processes/threads often share memory, files, or I/O resources.
- Without controlled access, some process may see inconsistent data.
- Without synchronization, results are often not deterministic and not reproducible.

Exam phrase

The important problem is not only that multiple processes exist. The problem is that their operations may be interleaved in different orders while accessing shared data.

The echo() shared global variable example

```
void echo() {
    chin = getchar();
    chout = chin;
    putchar(chout);
}
```

- The procedure is shared in memory to save space, but it uses shared global variable chin.
- P1 calls echo() and reads x into chin, then is interrupted.
- P2 calls echo(), reads y, and finishes, so chin becomes y.
- P1 resumes and uses chin, but chin is now y, not x. P1 prints the wrong character.
- The same problem can occur on a multiprocessor: P1 and P2 may run truly in parallel and still overwrite shared data.

Fix

Enforce single access: only one process can enter echo() at a time. Protect the code that accesses the shared variable/resource.

Race condition

- A race condition occurs when multiple processes/threads read/write shared data concurrently and the final result depends on their execution order.
- The output depends on who finishes last. Example: if P1 writes a = 1 and P2 writes a = 2, the final value depends on which write happens last.
- Even if two processes update different variables, the result can still depend on order if the variables are related.

```
static int b = 1, c = 2;
P3: b = b + c;
```

```
P4: c = b + c;

If P3 first: b = 3, then P4 makes c = 5.
If P4 first: c = 3, then P3 makes b = 4.
```

2. Mutual exclusion and the critical section problem

- A critical resource is a single non-shareable resource, such as a printer or shared object.
- A critical section is the code portion that accesses the critical resource.
- Only one process can be allowed inside its critical section for a given resource at a time.
- The OS cannot always know the logical requirements of the programmer, so processes must express their need for mutual exclusion. The OS provides support such as locks.

```
do {
    entry section       // ask permission
    critical section    // access shared resource
    exit section        // release permission
    remainder section   // normal code
} while (true);
```

Mutual exclusion mechanism

- For a shared resource Ra, every process wraps its critical section using entercritical(Ra) and exitcritical(Ra).
- If another process is already inside the critical section for the same resource, the requester must wait.
- The argument can represent a variable, file, device, or any shared object.

```
while (true) {
    /* preceding code */
    entercritical(Ra);
    /* critical section using Ra */
    exitcritical(Ra);
    /* following code */
}
```

Requirements for a correct mutual exclusion solution

1. Mutual exclusion: only one process at a time may enter its critical section for the same resource/shared object.
2. A process halted in its noncritical section must not interfere with other processes.
3. No indefinite delay: no deadlock or starvation for a process needing the critical section.
4. If no process is inside the critical section, a requesting process should be allowed to enter without delay.
5. Do not assume anything about relative process speeds or number of processors.
6. A process stays inside its critical section for a finite time only.

Memorize these as

Only one inside - no interference outside - no deadlock/starvation - no unnecessary delay - no speed/CPU assumptions - finite critical section.

Deadlock vs starvation

Problem	Meaning	Tiny example
Deadlock	Processes wait forever because each is waiting for something held/needed by another.	P1 holds printer and waits for scanner; P2 holds scanner and waits for printer.
Starvation	A process is repeatedly denied access	Weak semaphore keeps choosing other

	even though the system as a whole continues.	processes, so one process waits indefinitely.

3. Low-level and hardware-based solutions

Disabling interrupts

- Preventing a process from being interrupted can guarantee mutual exclusion on a uniprocessor.
- Example: disable interrupts before updating balance; enable interrupts afterward.
- But this is dangerous and limited.

```
disableInterrupts();
balance = balance + amount;
enableInterrupts();
```

Disadvantage	Why it matters
Interrupts may be disabled too long	System responsiveness suffers.
User process could abuse the privilege	Should not be available to normal user programs.
Blocks unrelated processes	We only wanted to stop P1/P2 interference, not the whole system.
Fails on multiprocessors	Disabling interrupts on one CPU does not disable interrupts on another CPU.

Atomic hardware instructions

- Modern machines provide atomic instructions. Atomic means non-interruptible.
- The lecture focuses on test_and_set() and compare_and_swap().
- They implement locks by making the check-and-change step indivisible.

test_and_set()

```
boolean test_and_set(boolean *target) {
    boolean rv = *target;
    *target = TRUE;
    return rv;
}

Shared boolean lock = FALSE;
do {
    while (test_and_set(&lock)) ;   // busy wait
    /* critical section */
    lock = FALSE;
    /* remainder section */
} while (true);
```

- It returns the old value and sets the value to TRUE atomically.
- If lock was FALSE, test_and_set returns FALSE and the process enters.
- If lock was TRUE, it returns TRUE and the process keeps spinning.

compare_and_swap()

```
int compare_and_swap(int *value, int expected, int new_value) {
    int temp = *value;
    if (*value == expected)
        *value = new_value;
    return temp;
}
```

```
Shared int lock = 0;
do {
    while (compare_and_swap(&lock, 0, 1) != 0) ;
    /* critical section */
    lock = 0;
    /* remainder section */
} while (true);
```

- It changes value to new_value only if value equals expected.
- For the lock: 0 means free, 1 means locked.
- If compare_and_swap returns 0, you successfully acquired the lock.

Special machine instructions: pros and cons

Advantages	Disadvantages
Works for any number of processes on uniprocessor or multiprocessor systems sharing memory.	Uses busy waiting, so waiting processes consume CPU time.
Simple and easy to verify.	Starvation is possible because selection of a waiting process can be arbitrary.
Can support multiple critical sections by using one lock variable per critical section.	Deadlock is possible if locks are used badly.

Mutex locks and spinlocks

```
acquire() {
    while (!available) ;   // busy wait
    available = false;
}

release() {
    available = true;
}
```

- A mutex lock protects a critical section to prevent race conditions.
- A process calls acquire() before the critical section and release() after leaving.
- Calls to acquire() and release() must be atomic, usually implemented using hardware atomic instructions.
- If it uses busy waiting, it is called a spinlock because the process spins while waiting.
- Spinlocks waste CPU cycles, but avoid context-switch overhead. They are useful when locks are held for very short times, especially on multiprocessors.

4. Semaphores - the main exam topic

Definition

A semaphore is an integer value used for signaling among processes. It is accessed only through atomic operations: initialize, wait/semWait, and signal/semSignal.

Semaphore value	Meaning
S > 0	S processes can execute wait(S) and continue immediately. It can represent available resource instances.
S = 0	No immediate access remains; the next wait will block.
S < 0	The magnitude |S| equals the number of processes waiting to be unblocked.

Basic wait() and signal()

```
wait(S) {
    while (S <= 0) ;   // busy wait
    S--;
}

signal(S) {
    S++;
}
```

- wait(S) tries to enter/use a resource. It decrements the semaphore after access is allowed.
- signal(S) releases/announces availability. It increments the semaphore.
- These operations must be atomic. If two processes execute wait/signal on the same semaphore at the same time, the semaphore itself becomes a race condition.

Counting vs binary semaphore vs mutex

Type	Value range	Use	Important detail
Counting semaphore	Unrestricted integer	Control access to a finite number of resource instances.	Initialize to number of available instances.
Binary semaphore	0 or 1	Mutual exclusion or event signaling.	Can be signaled by a different process from the one that waited.
Mutex lock	Locked/unlocked	Mutual exclusion only.	Normally the same thread/process that locks it must unlock it.

Semaphore for ordering: S1 before S2

- Use a semaphore initialized to 0 when P2 must wait until P1 finishes statement S1.

```
semaphore synch = 0;

P1:             P2:
S1;             wait(synch);
signal(synch);  S2;
```

Semaphore implementation with no busy waiting

- Each semaphore has a waiting queue/list.
- block() places the process on the semaphore waiting queue.
- wakeup(P) removes a process from the waiting queue and places it in the ready queue.

```
typedef struct {
    int value;
    struct process *list;
} semaphore;

wait(semaphore *S) {
    S->value--;
    if (S->value < 0) {
        add this process to S->list;
        block();
    }
}

signal(semaphore *S) {
    S->value++;
```

```
    if (S->value <= 0) {
        remove a process P from S->list;
        wakeup(P);
    }
}
```

Binary semaphore primitives

```
struct binary_semaphore {
    enum {zero, one} value;
    queueType queue;
};

semWaitB(S):
    if S.value == one: S.value = zero
    else: put process in S.queue and block it

semSignalB(S):
    if S.queue is empty: S.value = one
    else: remove one process from S.queue and make it ready
```

Mutual exclusion using semaphores

```
semaphore s = 1;

void P(int i) {
    while (true) {
        semWait(s);
        /* critical section */
        semSignal(s);
        /* remainder */
    }
}
```

- Initialize s to 1 because only one process can enter.
- The first process makes s = 0 and enters.
- Other processes executing semWait(s) will block and queue.
- semSignal(s) releases one waiting process if any.

Strong vs weak semaphores

Semaphore type	Queue rule	Starvation?
Strong semaphore	FIFO: the process blocked the longest is released first.	Guarantees freedom from starvation.
Weak semaphore	No specified order of removal from waiting queue.	Starvation is possible.

5. Classic synchronization problems

A. Producer/Consumer problem

- Producers generate items and place them in a buffer.
- Consumers remove items from the buffer one at a time.
- Only one producer/consumer should access the buffer at a time because b[], in, and out are shared.
- Producer must not insert into a full buffer.
- Consumer must not remove from an empty buffer.

Infinite buffer idea

```
Producer:
while (true) {
    produce item v;
    b[in] = v;
    in++;
}

Consumer:
while (true) {
    while (in <= out) ;   // wait if empty
    w = b[out];
    out++;
    consume item w;
}
```

- Problem: b[], in, and out are shared. Mutual exclusion is still needed.

Binary semaphore attempt and its flaw

- s enforces mutual exclusion.
- delay forces the consumer to wait if the buffer is empty.
- n is the number of items in the buffer: n = in - out.
- The lecture shows this program has a timing flaw: the consumer can act using stale knowledge of n if the order of operations interleaves badly.
- Fix: introduce local variable m in the consumer critical section and test m later, after leaving the critical section.

More elegant producer/consumer with counting semaphore n

```
semaphore n = 0;  // number of items in buffer
semaphore s = 1;  // mutual exclusion for buffer access

Producer:
while (true) {
    produce();
    semWait(s);
    append();
    semSignal(s);
    semSignal(n);
}

Consumer:
while (true) {
    semWait(n);
    semWait(s);
    take();
    semSignal(s);
    consume();
}
```

Professor trap: order matters

If consumer reverses semWait(n) and semWait(s), deadlock can occur: consumer enters the critical section when buffer is empty and prevents any producer from appending.

If producer reverses semSignal(s) and semSignal(n), no deadlock: consumer still needs both semaphores before proceeding.

Bounded buffer with semaphores

- The buffer is finite/circular.
- mutex = 1 protects the buffer itself.

- full = 0 counts full slots/items available to consume.
- empty = n counts empty slots available to produce into.

```
// n buffers, each holds one item
semaphore mutex = 1;
semaphore full = 0;
semaphore empty = n;

Producer:
do {
    produce next_produced;
    wait(empty);     // need empty slot first
    wait(mutex);     // then enter buffer critical section
    add item to buffer;
    signal(mutex);
    signal(full);    // one more full slot
} while (true);

Consumer:
do {
    wait(full);      // need item first
    wait(mutex);     // then enter buffer critical section
    remove item from buffer;
    signal(mutex);
    signal(empty);   // one more empty slot
    consume item;
} while (true);
```

B. Readers-Writers problem

- A shared data area is used by readers and writers.
- Readers only read; writers modify.
- Multiple readers may read simultaneously.
- Only one writer may write at a time.
- If a writer is writing, no reader may read.
- Writers must exclude all other processes: readers and writers.

Why not simple mutual exclusion?

If every read is a critical section with one-at-a-time access, readers become unnecessarily serialized. That is correct but too slow.

Why producer/consumer is not just readers/writers

The producer and consumer both read and modify queue pointers, so they are not pure writer/reader roles.

Readers priority solution

- wsem enforces writer/data-area mutual exclusion.
- readcount counts active readers.
- x protects readcount.
- First reader waits on wsem. Later readers can enter without waiting on wsem.
- Last reader signals wsem.
- Problem: writers may starve because as long as readers keep arriving, writers may never get access.

Writers priority solution

- Goal: once at least one writer wants access, no new readers are allowed into the data area.
- rsem inhibits readers while a writer is waiting/desiring access.
- writecount counts writers wanting access.
- y protects writecount.
- z prevents a long reader queue from building up on rsem; only one reader queues on rsem and additional readers queue on z.
- wsem is still used to enforce exclusive access to the shared data.

System state	Queue/lock idea
Readers only	wsem is set; no queues.
Writers only	wsem and rsem set; writers queue on wsem.
Readers + writers, read first	wsem set by reader, rsem set by writer, writers queue on wsem, one reader queues on rsem, other readers queue on z.
Readers + writers, write first	wsem set by writer, rsem set by writer, writers queue on wsem, one reader queues on rsem, other readers queue on z.

C. Dining philosophers problem

- Five philosophers alternate between thinking and eating.
- Each needs two chopsticks to eat: left and right.
- No two neighboring philosophers can use the same chopstick at the same time.
- The goal is to avoid both deadlock and starvation.

```
semaphore chopstick[5] = {1,1,1,1,1};

Philosopher i:
do {
    wait(chopstick[i]);
    wait(chopstick[(i+1) % 5]);
    eat;
    signal(chopstick[i]);
    signal(chopstick[(i+1) % 5]);
    think;
} while (true);
```

Problem with this naive algorithm

If all philosophers pick up their left chopstick at the same time, all chopsticks become unavailable. Each then waits forever for the right chopstick. That is deadlock, and starvation may also occur.

- One simple deadlock-avoidance idea from the lecture: add an attendant who allows only four philosophers in the dining room at once.
- With at most four seated philosophers, at least one philosopher can obtain two chopsticks.

6. Problems with semaphores

- Semaphores are powerful, but small mistakes create timing errors that are hard to detect because they occur only under certain interleavings.
- If signal(mutex) and wait(mutex) are interchanged around the critical section, multiple processes may enter the critical section simultaneously.

- If signal(mutex) is replaced by wait(mutex), deadlock can occur.
- If wait(mutex) or signal(mutex) is omitted, either mutual exclusion is violated or deadlock occurs.
- Because of these risks, higher-level constructs such as monitors were developed.

7. Monitors

Definition

A monitor is a high-level programming-language construct/ADT that provides mutual exclusion automatically for its procedures and local shared data.

- A monitor contains local data, initialization code, and one or more procedures.
- Local data variables are accessible only inside the monitor.
- A process enters the monitor by calling one of its procedures.
- Only one process may execute inside the monitor at a time; others wait in the monitor entry queue.
- This reduces programmer error because the programmer does not write the mutual-exclusion lock manually.

```
monitor monitor_name {
    // shared/local variable declarations
    procedure P1(...) { ... }
    procedure Pn(...) { ... }
    initialization_code(...) { ... }
}
```

Condition variables

- Condition variables provide synchronization inside monitors.
- They are special data types accessible only by the monitor.
- Declare them like: condition x, y;
- x.wait(): the calling process is suspended until x.signal(). It releases the monitor so another process can enter.
- x.signal(): resumes one process waiting on x, if any.
- If no process is waiting on x, the signal is lost. This is different from semaphore signal, which changes the semaphore value.

Signal choices inside monitors

Choice	Meaning
Signal and wait	P signals Q, then P waits until Q leaves the monitor or waits on another condition.
Signal and continue	P keeps running; Q waits until P leaves the monitor or waits on another condition.
Concurrent Pascal compromise	P executes signal and immediately leaves the monitor; Q resumes.
Mesa/C#/Java style	Generally closer to signal-and-continue behavior; signaled process rechecks condition when it runs.

Bounded buffer using a monitor

- Producer can add to buffer only by calling append(x) inside the monitor.

- Consumer can remove from buffer only by calling take(x) inside the monitor.
- notfull blocks producers when buffer is full.
- notempty blocks consumers when buffer is empty.
- The monitor automatically prevents two processes from modifying count/nextin/nextout at the same time.

```
monitor boundedbuffer {
    char buffer[N];
    int nextin, nextout, count;
    condition notfull, notempty;

    void append(char x) {
        if (count == N) wait(notfull);
        buffer[nextin] = x;
        nextin = (nextin + 1) % N;
        count++;
        signal(notempty);
    }

    void take(char x) {
        if (count == 0) wait(notempty);
        x = buffer[nextout];
        nextout = (nextout + 1) % N;
        count--;
        signal(notfull);
    }

    initialization: nextin = 0; nextout = 0; count = 0;
}
```

Dining philosophers using a monitor

- The monitor keeps state[i] as THINKING, HUNGRY, or EATING.
- Each philosopher has a condition variable self[i].
- pickup(i) sets philosopher i to HUNGRY and calls test(i). If not eating, the philosopher waits on self[i].
- putdown(i) sets philosopher i to THINKING and tests left and right neighbors.
- test(i) allows philosopher i to eat only if i is hungry and both neighbors are not eating.
- This ensures no two neighbors eat simultaneously and avoids deadlock, but starvation is still possible.

Monitor implementation using semaphores

- mutex = 1 protects entry to the monitor.
- next = 0 is used when a signaling process must wait for the resumed process.
- next_count counts processes suspended on next.
- For each condition variable x: x_sem = 0 and x_count = 0.

```
Each monitor procedure F:
wait(mutex);
...
body of F;
...
if (next_count > 0)
    signal(next);
else
    signal(mutex);

x.wait:
x_count++;
```

```
if (next_count > 0) signal(next);
else signal(mutex);
wait(x_sem);
x_count--;

x.signal:
if (x_count > 0) {
    next_count++;
    signal(x_sem);
    wait(next);
    next_count--;
}
```

8. Message passing and process interaction

- Process interaction requires communication and synchronization.
- Message passing provides both and works in shared-memory and distributed systems.
- The two core primitives are send(destination, message) and receive(source, message).
- Crucial synchronization point: a message can be consumed only once.

```
send(destination, message)
receive(source, message)
```

Synchronization options in message passing

Option	Meaning	Exam phrase
Blocking send + blocking receive	Both sender and receiver wait until message is delivered.	Rendezvous; tight synchronization.
Non-blocking send + non-blocking receive	Neither side is required to wait.	Most flexible but less synchronized.
Non-blocking send + blocking receive	Sender continues; receiver waits until message arrives.	Most useful combination in many systems.

Addressing

Type	Meaning
Direct addressing	send names the destination process explicitly. receive may explicitly name sender or use implicit addressing to learn the source.
Indirect addressing	Messages go through a shared queue/mailbox. Sender sends to mailbox; receiver receives from mailbox.
Mailbox/port	A temporary message queue. Many-to-one mailbox is often called a port in client/server style.

Relationships between senders and receivers

Relationship	Use
One-to-one	Private communication link between two processes; reduces interference.
Many-to-one	Many clients send to one server; mailbox often called a port.
One-to-many	Broadcast-style communication: one sender, multiple receivers.
Many-to-many	Multiple servers provide service to multiple clients.

General message format

- Message format depends on the messaging facility objectives.
- Short fixed-length messages reduce processing/storage overhead.
- Variable-length messages give flexibility.
- Header may contain source id, destination id, length, type, control information, pointer for linked list of messages, sequence number, and priority.
- Body contains the message contents.

Mutual exclusion using messages

- Use non-blocking send and blocking receive.
- All concurrent processes share a mailbox called box, initialized with a single message.
- To enter the critical section, a process receives the message from box.
- If box is empty, the process blocks.
- After leaving the critical section, the process sends the message back to box.
- The message acts like a token. Only the token holder enters the critical section.

> **Assumption**
>
> If multiple processes receive concurrently, only one gets the message. If the queue is empty, all block; when a message arrives, only one waiting process is activated and receives it.

Bounded buffer using messages

- Use two mailboxes: mayproduce and mayconsume.
- mayproduce initially receives capacity empty messages/tokens.
- Producer must receive from mayproduce before producing; then it sends the produced item to mayconsume.
- Consumer receives item from mayconsume, consumes it, then sends an empty message back to mayproduce.
- The empty messages act like the empty semaphore; produced messages act like the full semaphore.

```
Producer:
receive(mayproduce, pmsg);   // wait for empty slot token
pmsg = produce();
send(mayconsume, pmsg);      // item available

Consumer:
receive(mayconsume, cmsg);   // wait for item
consume(cmsg);
send(mayproduce, null);      // empty slot returned

main:
create_mailbox(mayproduce);
create_mailbox(mayconsume);
for i = 1 to capacity: send(mayproduce, null);
parbegin(producer, consumer);
```

Writers priority using message passing

- A controller process has access to the shared data area.
- Readers and writers send request messages to the controller.

- Controller grants access by sending OK reply messages.
- After access, reader/writer sends a finished message.
- Controller has three mailboxes: readrequest, writerequest, and finished.
- Controller must enforce mutual exclusion and service write requests before read requests.
- Variable count is initialized to a number greater than the maximum possible readers, e.g. 100.
- If count > 0: no writer is waiting; service finished first, then writers, then readers.
- If count = 0: a writer is outstanding; allow writer to proceed and wait for finished.
- If count < 0: writer is waiting for active readers to finish; service only finished messages.

9. OS and API synchronization examples

System/API	Synchronization mechanisms to know
Windows	Interrupt masks on uniprocessors; spinlocks on multiprocessors; dispatcher objects such as mutexes, semaphores, events, and timers. Dispatcher objects have signaled/non-signaled states.
Linux	Before kernel 2.6: interrupts disabled for short critical sections. 2.6+: fully preemptive. Provides atomic integers, semaphores, spinlocks, and reader-writer versions. Single-CPU spinlocks replaced by enabling/disabling kernel preemption.
UNIX	Pipes, messages, and shared memory for communication; semaphores and signals for triggering/synchronization.
Solaris	Adaptive mutexes, condition variables, readers-writers locks, turnstiles, priority inheritance. Adaptive mutex spins if holder is running on another CPU, otherwise blocks/sleeps.
Pthreads	OS-independent API. Provides mutex locks and condition variables; non-portable extensions include read-write locks and spinlocks.

10. Master comparison table

Mechanism	Main use	Strength	Weakness / trap
Disabling interrupts	Mutual exclusion on uniprocessor kernel code.	Simple idea.	Unsafe for user processes; fails on multiprocessors.
test_and_set / compare_and_swap	Build low-level locks atomically.	Works on multiprocessors sharing memory.	Busy waiting; starvation/deadlock possible.
Mutex / spinlock	Protect one critical section/resource.	Simple and fast for short critical sections.	Busy waiting wastes CPU; must be acquired/released correctly.
Semaphore	Mutual exclusion and general synchronization/counting resources.	Very powerful and flexible.	Easy to misuse; deadlock/starvation/timing bugs.
Monitor	High-level safe access to shared data.	Automatic mutual exclusion; easier to reason about.	Needs condition variables for blocking conditions; signal semantics matter.
Message passing	Communication plus synchronization, including distributed systems.	No shared memory required; messages can be tokens.	Need addressing/mailbox semantics; blocking choices matter.

11. High-yield exam traps

1. Race condition is about final result depending on order, not just about processes running at the same time.
2. Critical section is code, not the resource itself. Critical resource is the shared/non-shareable thing.
3. Mutual exclusion must be enforced only for processes that share the same resource/object.
4. Disabling interrupts works only for uniprocessor mutual exclusion and is dangerous for user code.
5. Atomic instructions do not remove busy waiting by themselves.
6. Spinlock wastes CPU but avoids context switching; useful for short waits on multiprocessors.
7. Binary semaphore is similar to mutex, but unlike a mutex it may be signaled by a different process.
8. Negative semaphore value means number of blocked processes.
9. Strong semaphore = FIFO = no starvation. Weak semaphore = no specified order = possible starvation.
10. In bounded buffer, producer waits on empty before mutex; consumer waits on full before mutex.
11. In readers-writers, readers can share with readers, but writers exclude everyone.
12. Readers-priority solution can starve writers. Writers-priority solution blocks new readers once a writer is waiting.
13. Dining philosophers naive solution deadlocks if everyone picks left chopstick first.
14. Monitor signal is lost if nobody is waiting; semaphore signal is remembered by increasing the semaphore value.
15. Message passing token in mailbox can implement mutual exclusion.

12. Likely exam questions with answers

Q: Define concurrency.

A: Concurrency is the management of interacting processes/threads whose executions may overlap. It includes communication, resource sharing, synchronization, and CPU allocation.

Q: What is a race condition?

A: It occurs when multiple processes/threads access shared data concurrently and the final result depends on execution order.

Q: What is mutual exclusion?

A: It ensures that if one process is accessing a shared critical resource, others are excluded from accessing it at the same time.

Q: What is the critical section problem?

A: Designing a protocol so processes cooperate: request entry, execute critical section exclusively, exit, then continue remainder code.

Q: List the six requirements for mutual exclusion.

A: Only one inside; no interference if halted outside; no deadlock/starvation; no delay when section empty; no speed/CPU assumptions; finite time in critical section.

Q: Why is disabling interrupts not enough?

A: It can be abused, blocks unrelated processes, can last too long, and does not guarantee mutual exclusion on multiprocessors.

Q: Explain test_and_set.

A: It atomically returns the old value of a boolean and sets it to TRUE. Used to acquire a lock by spinning while old value is TRUE.

Q: Explain compare_and_swap.

A: It atomically compares a memory value to an expected value and updates it only if equal, returning the old value.

Q: What is busy waiting?

A: A process repeatedly checks a condition/lock while consuming CPU instead of blocking.

Q: What is a semaphore?

A: An integer synchronization variable accessed only using atomic wait and signal operations.

Q: What does S < 0 mean for a semaphore?

A: The absolute value is the number of processes blocked/waiting on that semaphore.

Q: Counting vs binary semaphore?

A: Counting semaphore can take many integer values and controls multiple resource instances; binary semaphore is only 0/1 and can act like a mutex/event.

Q: Mutex vs binary semaphore?

A: Mutex is usually unlocked by the same thread that locked it; a binary semaphore can be signaled by a different process.

Q: Why can readers read together but writers cannot?

A: Readers do not modify shared data, so multiple readers are safe; writers modify data and must exclude all readers/writers.

Q: Why can readers-priority starve writers?

A: Continuous arrival of readers can keep the data area under reader control, preventing writers from entering.

Q: What is the dining philosophers deadlock?

A: If every philosopher picks up the left chopstick first, each waits forever for the right chopstick.

Q: What is a monitor?

A: A high-level ADT/software module with local data/procedures that enforces one active process inside the monitor at a time.

Q: How do condition variables work?

A: x.wait suspends the caller and releases the monitor; x.signal wakes one process waiting on x if any.

Q: Semaphore signal vs monitor signal?

A: Semaphore signal is remembered by increasing the value; monitor condition signal is lost if no process is waiting.

Q: What is rendezvous?

A: Blocking send plus blocking receive: both sender and receiver wait until message delivery completes.

Q: How can messages implement mutual exclusion?

A: Initialize a mailbox with one token message. A process receives the token to enter the critical section and sends it back after exiting.

13. Ultra-fast flashcards

Term	Answer
Critical resource	Single non-shareable resource.
Critical section	Code that accesses a critical resource.
Entry section	Requests permission to enter critical section.
Exit section	Releases permission/lock after critical section.
Atomic	Indivisible/non-interruptible.
Deadlock	Processes wait forever for each other.
Starvation	Process denied access indefinitely.
Spinlock	Busy-waiting lock.
semWait/wait	Decrement; may block.
semSignal/signal	Increment; may wake one blocked process.
Strong semaphore	FIFO waiting queue.
Weak semaphore	Unspecified waiting order.
full semaphore	Counts full buffer slots/items.

empty semaphore	Counts empty buffer slots.
readcount	Number of active readers.
writecount	Number of writers wanting access.
condition variable	Monitor-only variable for wait/signal synchronization.
rendezvous	Blocking send + blocking receive.
mailbox	Indirect message queue/port.
token message	Single message that grants critical-section access.

Final 10-minute memorization line

Race condition happens when timing/order changes results. Critical sections protect critical resources. Correct mutual exclusion means one inside, no deadlock/starvation, no assumptions, finite time. test_and_set and compare_and_swap create locks but may busy-wait. Semaphores use atomic wait/signal; negative count means waiting processes. Bounded buffer uses empty/full/mutex. Readers can share; writers exclude. Monitors give automatic mutual exclusion plus condition variables. Message passing can synchronize by blocking and can implement mutual exclusion with a token.